

머신러닝

선형회귀 심화

Chapter 8



Prof. Yang



한국성서대학교
KOREAN BIBLE UNIVERSITY

목 차

CONTENTS

01 경사하강법의 종류

02 과대적합과 정규화

03 사이킷런을 이용한 선형회귀

01

경사하강법의 종류



복습: 경사하강법

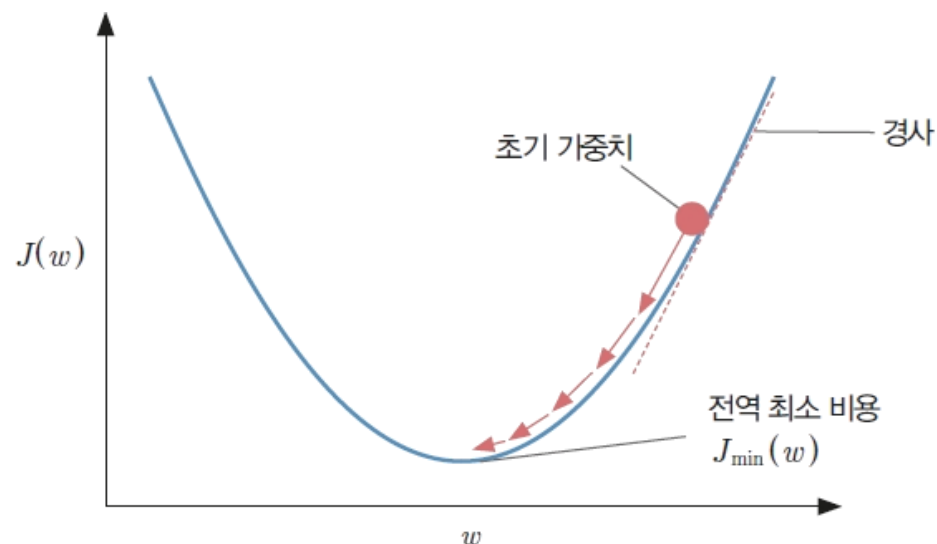
- 비용함수 $f(x) = (\hat{y}^{(i)} - y^{(i)})^2$ 는 최소화시켜야 하는 값이고 $2x$ 는 이를 미분한 값인 경사

$$f(x) = x^2 \rightarrow \frac{dy}{dx} = 2x$$

- 경사하강법의 기본 수식

$$x_{new} = x_{old} - \alpha \times \overset{\text{경사값}}{(2x_{old})}$$

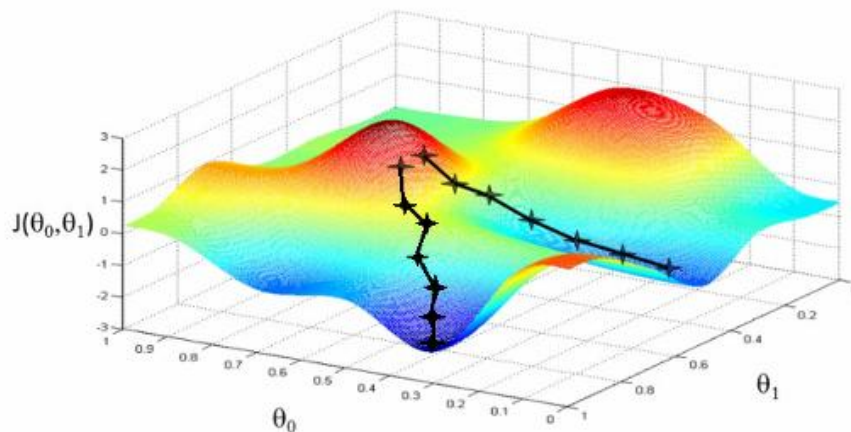
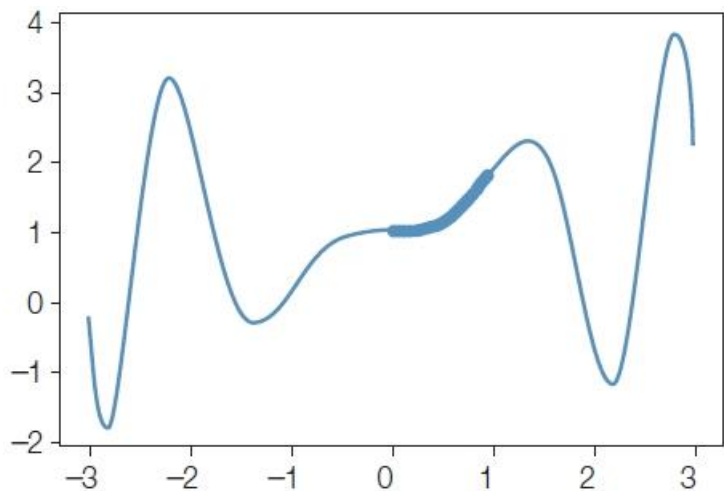
- x_{old} 는 현재의 x 값, x_{new} 는 경사 값이 적용된 후 생성된 값
- 경사만큼의 변화가 계속 x 에 적용되어 x 의 최솟값 찾을
 - 반복적으로 미분 값을 적용시키면서 더 이상 값이 변하지 않거나 변화가 미미해지는 지점까지 값이 줄어듦





전체-배치 경사하강법

- 전체-배치 경사하강법(full-batch gradient descent) : 모든 데이터를 한 번에 입력하는 경사하강법
 - 배치(batch) : 한 번의 학습에서 사용되는 데이터 묶음
- 전체 데이터를 사용하여 가중치를 업데이트할 때의 단점
 - 계산량이 많아 시간이 오래 걸림
 - 초기값에 따라 지역 최적점(local minimum)에 수렴할 수 있음





전체-배치 경사하강법의 특징

- 업데이트 횟수 감소 : 가중치 업데이트 횟수가 줄어 계산상 효율성 상승
- 안정적인 비용함수 수렴 : 모든 데이터의 평균 경사를 사용하므로 비교적 안정적으로 수렴
- 업데이트 속도 증가 : 대규모 데이터셋을 한 번에 처리하면 모델의 매개변수 업데이트 속도에 문제 발생이 적음
- 데이터가 백만 단위 이상을 넘어가면 하나의 머신에서는 처리가 불가능해져서 메모리 문제가 발생
- 사이킷런에서는 전체-배치 경사하강법을 지원하는 함수는 없음
 - 전체-배치 경사하강법 코드는 Ch07.선형회귀기초 p.32 참고



확률적 경사하강법(SGD)

- 확률적 경사하강법(Stochastic Gradient Descent, SGD)
: 학습용 데이터에서 샘플들을 랜덤하게 뽑아서 사용
- 대상 데이터를 섞은(shuffle) 후, 일반적인 경사하강법처럼 데이터를 한 개씩 추출하여 가중치 업데이트

```
1 Procedure SGD
2   shuffle(X)
3   for i in number of X do
4      $\theta_j = \theta_j - \alpha(\hat{y}^{(i)} - y^{(i)})x_j^{(i)}$ 
5   end for
6 end procedure
```

그림 8-2 확률적 경사하강법(SGD) 알고리즘의 의사코드



확률적 경사하강법(SGD) 특징

- SGD의 장점
 - 빈번한 업데이트를 하기 때문에 데이터 분석가가 모델의 성능 변화를 빠르게 확인
 - 데이터 특성에 따라 빠르게 결과에 도달할 수 있음
 - 지역 최적화를 회피
- SGD의 단점
 - 대용량 데이터를 사용하는 경우 시간이 매우 오래 걸림
 - 결과의 마지막 값을 확인하기 어려움
 - 흔히 '튀는 현상'이라고 불리는데 비용함수의 값이 줄어들지 않고 계속 변화할 때 정확히 언제 루프가 종료되는지 알 수 없어 판단이 어렵다



확률적 경사하강법(SGD) 코드

```
import numpy as np
from sklearn.linear_model import SGDRegressor
from sklearn.utils import shuffle

X = np.random.uniform(0, 2, 100).reshape(-1, 1) # 0~2 사이 난수
noise = np.random.normal(0, 1, 100) # 평균0 표준편차1 노이즈

# 3x+4+노이즈
y = 3 * X[:,0] + 4 + noise

# 3. SGD 모델 생성
model = SGDRegressor(
    loss="squared_error",
    shuffle=True)

# 4. 학습
model.fit(X, y)

print("가중치:", model.coef_)
print("절편:", model.intercept_)
```



가중치: [2.93971139]
절편: [4.04951618]



SGDRegressor 객체

파라미터	설명
loss	손실 함수 지정 (기본: squared_error → MSE)
penalty	정규화 방식 : l1, l2, elasticnet
alpha	정규화 강도 (λ 역할)
learning_rate	학습률 변화 방식
	constant 학습률을 항상 동일하게 유지
	optimal 자동으로 감소하는 학습률
	invscaling 반복할수록 점점 감소
	adaptive 성능이 개선되지 않으면 감소
eta0	초기 학습률(숫자)
max_iter	epoch 횟수
shuffle	매 epoch마다 데이터 셔플 여부

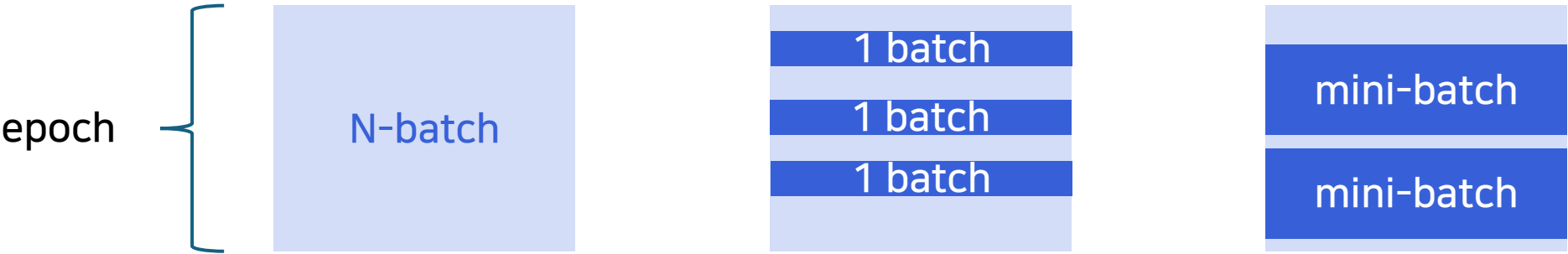


미니-배치 경사하강법

- 미니-배치 경사하강법(mini-batch gradient descent) 또는 미니-배치 SGD(mini-batch SGD)
: 데이터의 랜덤한 일부분만 입력해서 경사도 평균을 구해 가중치 업데이트
- 에포크(epoch) : 데이터를 한 번에 모두 학습시키는 횟수
 - 전체-배치 SGD를 한 번 학습하는 루프가 실행될 때 1에포크의 데이터가 학습된다고 말함
- 배치 사이즈(batch-size) : 한 번에 학습되는 데이터의 개수
 - 총 데이터가 5012개 있고 배치 사이즈가 512라면 10번의 루프가 돌면서 1에포크를 학습했다고 말함
 - 마지막 배치는 512보다 작을 수 있음
- 에포크와 배치 사이즈는 하이퍼 매개변수이므로 데이터 분석가가 직접 선정함



경사하강법 비교



구분	전체-배치 경사하강법	확률적 경사하강법(SGD)	미니배치 경사하강법
배치 크기 (Batch Size)	전체 데이터 수 (N)	1	1보다 크고 (N)보다 작은 값
한 번의 Iteration 의미	전체 데이터를 한 번 사용	샘플 1개 사용	배치 1개 사용
1 Epoch당 Iteration 수	1	N	N / batch size
가중치 업데이트 빈도	가장 적음	가장 많음	중간
연산 속도	1회 업데이트는 느림	1회 업데이트는 빠름	중간
학습 안정성	안정적	불안정하고 흔들릴 수 있음	비교적 안정적
메모리 사용량	큼	작음	중간
대표 특징	정확한 gradient 계산	매우 자주 업데이트	실제로 가장 많이 사용됨

02

과대적합과 정규화



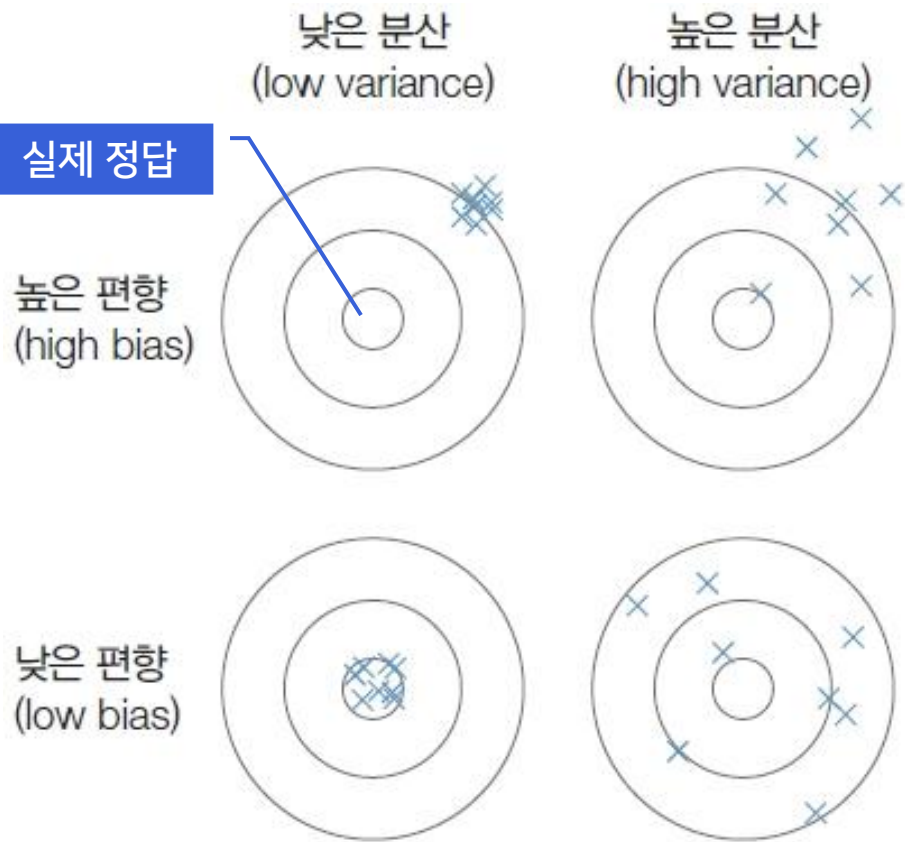
과대적합 극복하기

- 편향(bias) : 학습된 모델이 학습 데이터에 대해 만들어 낸 예측값과 실제값과의 차이
 - 모델의 결과가 얼마나 한쪽으로 쏠려 있는지 나타냄
 - 편향이 크면 학습이 잘 진행되기는 했지만 해당 데이터에만 잘 맞음
- 분산(variance) : 학습된 모델이 테스트 데이터에 대해 만들어 낸 예측값과 실제값과의 차이
 - 모델의 결과가 얼마나 퍼져 있는지 나타냄
- 편향-분산 트레이드오프(bias-variance trade-off) : 편향과 분산의 상충관계



과대적합 극복하기

실제 정답



- 편향: 훈련 데이터 예측값-실제값
- 분산: 테스트 데이터 예측값- 실제값

■ 과대적합(overfitting)

- 높은 분산 낮은 편향 상태
- 함수가 훈련 데이터셋에만 맞음
- 해결안: 안피쳐의 개수를 줄이거나 정규화

■ 과소적합(underfitting)

- 낮은 분산 높은 편향 상태
- 모델이 너무 단순하여 데이터의 패턴을 학습하지 못함
- 해결안
 - 모델 복잡도 감소(예. 피쳐 개수 줄이기)
 - 조기 종료(Early Stopping)



Epoch 증가에 따른 과대적합 현상

- 과대적합이 발생할 때 경사하강법 루프가 진행될수록 학습 데이터셋에 대한 비용함수의 값은 줄어들이지만 테스트 데이터셋의 비용함수 값은 증가

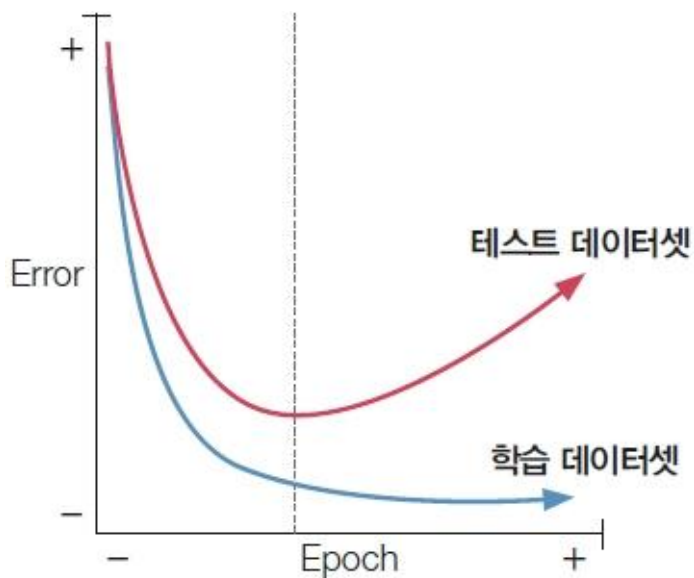


그림 8-5 과대적합이 발생할 때의 경사하강법

- 선형회귀 외에도 결정트리 알고리즘이나 딥러닝처럼 연산에 루프가 필요한 모든 알고리즘에서 똑같이 발생



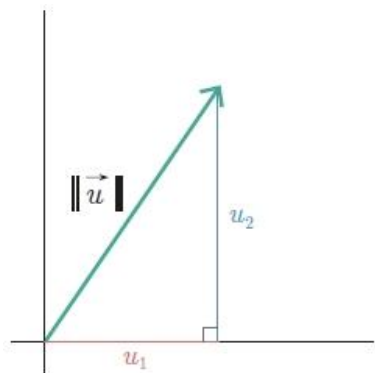
선형회귀에서 과대적합 해결책

- 오컴의 면도날 원리 : '보다 적은 수의 논리로 설명이 가능한 경우, 많은 수의 논리를 세우지 않는다'
 - 복잡한 설명 < 간단한 설명
 - '경제성의 원리' 또는 '단순성의 원리'
 - 머신러닝에서는 너무 많은 피처를 사용하지 않는 것
- 선형회귀에서 과대적합 해결책
 - 더 많은 데이터 활용하기 : 오류가 없고, 분포가 다양한 데이터를 많이 확보
 - 피처의 개수 줄이기 : 필요한 피처만 잘 찾아 사용
 - 적절한 매개변수 선정하기 : SGD의 학습률이나 루프의 횟수처럼 적절한 하이퍼 매개변수를 선정
 - 정규화 적용하기 : 데이터 편향성에 따라 필요 이상으로 증가한 피처의 가중치 값을 적절히 줄이는 규제 수식을 추가



L2 정규화 : 리지 회귀

- 리지 회귀(ridge regression) : L2 정규화(L2 regularization)라고 부름
- 놈(norm) : 좌표평면의 원점에서 점까지의 거리를 나타내어 벡터의 크기를 측정하는 기법



$$L_{2\text{norm}} = \|x\|_2 = \sqrt{(\sum_i^n |x|^2)} = (\sum_i^n |x_i|^2)^{\frac{1}{2}}$$

if $x = (x_1, x_2, \dots, x_n)$

- 벡터 각 원소들의 제곱합에 제곱근을 취함
- 리지 회귀는 L2 놈을 선형회귀의 비용함수 수식에 적용

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2} \sum_{j=1}^n \theta_j^2$$

- λ : 하이퍼 파라미터 상수로 개발자가 정해야함

- 뒷부분에 새로 붙인 수식은 페널티텀(penalty term)
- 모델의 가중치 값들의 제곱의 합
 - 가중치 값이 조금이라도 커질 때 비용함수에 매우 큰 영향
 - λ 가 클수록 전체 페널티텀의 값이 커져 θ 값이 조절됨
 - 리지 회귀 수식을 미분하면 j 의 값이 1 이상일 때 페널티가 적용



L1 정규화 : 라쏘 회귀

- 라쏘 회귀(lasso regression) L1 정규화(L1 regularization)라고 부름
- L1 놈(L1 norm) : 절대값을 사용하여 거리를 측정

$$\|x\|_1 := \sum_{i=1}^n |x_i|$$

- 가중치에 페널티를 추가하는데 기존 수식에다 L1 놈 페널티를 추가하여 계산

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2} \sum_{j=1}^n |\theta_j|$$

03

사이킷런을 이용한 선형회귀



사이킷런과 선형회귀 관련 함수

- 사이킷런(scikit-learn) : 대표적인 머신러닝 라이브러리

표 8-1 사이킷런의 선형회귀 관련 함수

함수명	설명	알고리즘
LinearRegression	가장 기본적인 선형회귀 알고리즘을 사용하며, SGD가 아닌 최소자승법으로 계산한다.	최소자승법
Lasso	L1 손실을 활용한 라쏘 알고리즘을 사용한다.	최소자승법
Ridge	L2 손실을 활용한 리지 알고리즘을 사용한다.	최소자승법
SGDRegressor	확률적 경사 하강법을 사용한 회귀 모델을 만든다. SGD에서 비용함수만을 변경하여 모든 함수를 지원하고 있어 필요한 하이퍼 매개변수를 설정해야 한다.	SGD

- 최소자승법과 SGD 기반 알고리즘 클래스 제공



데이터 설명

■ boston housing prices(보스턴 집값) 데이터셋

x 변수 13개	[01] CRIM	자치시(town)별 1인당 범죄율
	[02] ZN	25,000 평방피트를 초과하는 거주지역의 비율
	[03] INDUS	비소매상업지역이 점유하고 있는 토지의 비율
	[04] CHAS	찰스강에 대한 더미변수(강의 경계에 위치한 경우는 1, 아니면 0)
	[05] NOX	10ppm 당 농축 일산화질소
	[06] RM	주택 1가구당 평균 방의 개수
	[07] AGE	1940년 이전에 건축된 소유 주택의 비율
	[08] DIS	5개의 보스턴 직업센터까지의 접근성 지수
	[09] RAD	방사형 도로까지의 접근성 지수
	[10] TAX	10,000달러 당 재산세율
	[11] PTRATIO	자치시(town)별 학생/교사 비율
	[12] B	$1000(B_k - 0.63)^2$, 여기서 B_k 는 자치시별 흑인의 비율을 말함
	[13] LSTAT	모집단의 하위 계층의 비율(%)
y 변수	[14] MEDV	본인 소유의 주택 가격(중앙값) (단위: \$1,000)

그림 8-8 boston housing prices(보스턴 집값) 데이터셋



데이터 확보

```
import pandas as pd
```

```
#데이터 가져오기
```

```
data_url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/housing/housing.data'
```

```
data = pd.read_csv(data_url, sep='\s+', header = None) #정규 표현식: '\s+' 공백 하나 이상
```

```
#데이터의 열 이름 지정
```

```
data.columns = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT', 'MEDV']
```

```
data.head()
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296.0	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242.0	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242.0	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222.0	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222.0	18.7	396.90	5.33	36.2



x와 y 각 데이터셋을 추출

- y_data는 $n \times 1$ 의 형태로 변환

```
x_col = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT']
```

```
x_data = data[x_col].values #0~12번 열 추출후 넘파이 배열로 반환
```

```
#medv열을 넘파이 배열로 반환후, reshape함수 호출하여 행=자동 맞춤, 열=1 형태로 반환
```

```
y_data = data.MEDV.values.reshape(-1,1)
```

```
print(x_data.shape)
```

```
print(y_data.shape)
```

```
(506, 13)
```

```
(506, 1)
```




데이터 전처리 : 피쳐 스케일링

- 최솟값-최댓값 정규화(min-max normalization)

$$z_i = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$

Ch06 전처리 p.9 참고

```
from sklearn.preprocessing import MinMaxScaler
```

```
#fit:데이터의 최소값과 최대값을 학습하여 객체 반환
```

```
minmax_scale = MinMaxScaler(feature_range=(0,5)).fit(x_data)
```

```
#transform:학습된 기준을 이용하여 데이터를 스케일링한 값 반환
```

```
x_scaled_data = minmax_scale.transform(x_data)
```

```
#x_scaled_data=MinMaxScaler(feature_range=(0,5)).fit_transform(x_data)
```

```
x_scaled_data[:3]
```



```
array([[0.00000000e+00, 9.00000000e-01, 3.39076246e-01, 0.00000000e+00,
        1.57407407e+00, 2.88752635e+00, 3.20803296e+00, 1.34601570e+00,
        0.00000000e+00, 1.04007634e+00, 1.43617021e+00, 5.00000000e+00,
        4.48399558e-01],
       [1.17961270e-03, 0.00000000e+00, 1.21151026e+00, 0.00000000e+00,
        8.64197531e-01, 2.73998850e+00, 3.91349125e+00, 1.74480990e+00,
        2.17391304e-01, 5.24809160e-01, 2.76595745e+00, 5.00000000e+00,
        1.02235099e+00],
       [1.17848872e-03, 0.00000000e+00, 1.21151026e+00, 0.00000000e+00,
        8.64197531e-01, 3.47192949e+00, 2.99691040e+00, 1.74480990e+00,
        2.17391304e-01, 5.24809160e-01, 2.76595745e+00, 4.94868627e+00,
        3.17328918e-01]])
```



데이터 분리

- 데이터를 훈련과 테스트 형태로 분류

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(x_scaled_data, y_data, test_size=0.33)
```

```
# X 데이터의 학습 데이터셋, X 데이터의 테스트 데이터셋
```

```
# Y 데이터의 학습 데이터셋, Y 데이터의 테스트 데이터셋
```

```
X_train.shape, X_test.shape, y_train.shape, y_test.shape
```

```
((339, 13), (167, 13), (339, 1), (167, 1))
```



모델 객체 생성

- 학습에 사용할 알고리즘 해당하는 모델의 객체 생성
 - 각 클래스의 매개변수를 잘 사용해야함
 - 공통적으로 사용하는 매개변수
 - fit_intercept : 절편을 사용할지 말지를 선택
 - normalize : 학습할 때 값들을 정규화할지 말지
 - copy_X : 학습 시 데이터를 복사한 후 학습을 할지 결정
 - n_jobs : 연산을 위해 몇 개의 CPU를 사용할지 결정
 - alpha : 라쏘 회귀, 리지 회귀, SGD에 있음. 페널티 값을 지정

```
from sklearn import linear_model
```

```
regr = linear_model.LinearRegression(fit_intercept=True, copy_X=True, n_jobs=8)  
lasso_regr = linear_model.Lasso(alpha=0.01, fit_intercept=True, copy_X=True)  
ridge_regr = linear_model.Ridge(alpha=0.01, fit_intercept=True, copy_X=True)  
SGD_regr = linear_model.SGDRegressor(penalty="l2", alpha=0.01, max_iter=1000, tol=0.001, eta0=0.01)
```



데이터 학습

#최소자승법

```
regr.fit(X_train, y_train)
```

▼ LinearRegression ⓘ ?

```
LinearRegression(n_jobs=8)
```

#라쏘

```
lasso_regr.fit(X_train, y_train)
```

▼ Lasso ⓘ ?

```
Lasso(alpha=0.01)
```

#릿지

```
ridge_regr.fit(X_train, y_train)
```

▼ Ridge ⓘ ?

```
Ridge(alpha=0.01)
```

#SGD+릿지

```
SGD_regr.fit(X_train, y_train)
```

▼ SGDRegressor ⓘ ?

```
SGDRegressor(alpha=0.01)
```



학습 후 가중치 값

- 피쳐의 계수 coefficients값 확인
- 상수값(intercept) 확인

#기준 리니어

```
print('LinearRegression Coefficients: ', regr.coef_)  
print('LinearRegression intercept: ', regr.intercept_, "\n")
```

#라쏘

```
print('Lasso Coefficients: ', lasso_regr.coef_)  
print('Lasso intercept: ', lasso_regr.intercept_, "\n")
```

#리지

```
print('Ridge Coefficients: ', ridge_regr.coef_)  
print('Ridge intercept: ', ridge_regr.intercept_, "\n")
```

##SGD+릿지

```
print('SGD Coefficients: ', SGD_regr.coef_)  
print('SGD intercept: ', SGD_regr.intercept_)
```

```
LinearRegression Coefficients: [[-1.79776918  0.71933907  0.40185215  
-0.20552734 -3.10391998  1.28776054 -1.45492631  
-4.05463754]]
```

```
LinearRegression intercept: [24.81486406]
```

```
Lasso Coefficients: [-1.73478956  0.71933907  0.40185215  
-0.19428686 -3.03924224  1.22616664 -1.38412766 -  
-4.06164338]
```

```
Lasso intercept: [24.70955843]
```

```
Ridge Coefficients: [[-1.79747388  0.75456478  0.40185215  
-0.20550292 -3.1035298  1.28755629 -1.45470417  
-4.0546027 ]]
```

```
Ridge intercept: [24.81463431]
```

```
SGD Coefficients: [-1.11688694  0.61699876  0.87011215  
 0.20022333 -0.79476816  0.81655058 -1.07856918 -  
-3.09403029]
```

```
SGD intercept: [2.85502412]
```



예측값

■ 만들어진 함수로 예측값 생성

- 예측값을 생성하기 위한 X데이터에는 fit을 절대 쓰지 않고 transform만 사용해야 함
- Why? 정규화 과정은 train 데이터 기준으로 평균과 표준편차를 계산해야 함

```
#x데이터 정규화
reX = minmax_scale.transform(x_data)

regr.predict(reX[:5])
#lasso_regr.predict(reX[:5])
#ridge_regr.predict(reX[:5])
#SGD_regr.predict(reX[:5])
```



```
array([[29.87167237],
       [25.28727189],
       [30.9756791 ],
       [29.18743819],
       [28.43169418]])
```

- 예측값은 두 가지 방법 모두 같은 값을 출력
- 다른 모델들도 확인하기

■ 수식으로 예측값 생성

```
reX[:5].dot(regr.coef_.T) + regr.intercept_
#reX[:5].dot(lasso_regr.coef_.T) + lasso_regr.intercept_
#reX[:5].dot(ridge_regr.coef_.T) + ridge_regr.intercept_
#reX[:5].dot(SGD_regr.coef_.T) + SGD_regr.intercept_
```



```
array([[29.87167237],
       [25.28727189],
       [30.9756791 ],
       [29.18743819],
       [28.43169418]])
```



성능 평가: 테스트 데이터 사용

- 성능 평가는 학습에 사용되지 않은 새로운 데이터를 사용해야함
- 따라서 미리 분리해둔 테스트 데이터(X_{test}) 사용
- 학습된 각 회귀 모델을 이용하여 테스트 데이터의 예측값을 계산

```
scaled_x = minmax_scale.transform(X_test)
```

```
regr_y_hat = regr.predict(X_test)
```

```
lasso_y_hat = lasso_regr.predict(X_test)
```

```
ridge_y_hat = ridge_regr.predict(X_test)
```

```
SGD_y_hat = SGD_regr.predict(X_test)
```

- 테스트 데이터에 대한 정답 데이터(y_{test})는 원본 데이터 보호를 위해 복사하여 사용

```
y_true = y_test.copy()#정답
```



성능 평가: 선형회귀 대표적 성능 지표

- 사이킷런에서 지표들(metrics)을 호출하여 성능을 비교

```
from sklearn.metrics import r2_score
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error

#각 회귀분석 모델 평가: R2, MAE, MSE
print("LinearRegression평가:", r2_score(y_true, regr_y_hat),
      mean_absolute_error(y_true, regr_y_hat),
      mean_squared_error(y_true, regr_y_hat))

print("Lasso평가:", r2_score(y_true, lasso_y_hat),
      mean_absolute_error(y_true, lasso_y_hat),
      mean_squared_error(y_true, lasso_y_hat))

print("Ridge평가:", r2_score(y_true, ridge_y_hat),
      mean_absolute_error(y_true, ridge_y_hat),
      mean_squared_error(y_true, ridge_y_hat))

print("SGD평가:", r2_score(y_true, SGD_y_hat),
      mean_absolute_error(y_true, SGD_y_hat),
      mean_squared_error(y_true, SGD_y_hat))
```

LinearRegression평가: 0.7423892400920797 3.608436917652707 25.205965769296096
Lasso평가: 0.7443928173150511 3.5840003737082933 25.00992543729906
Ridge평가: 0.7424050079207941 3.6082609048377705 25.204422963588087
SGD평가: 0.6999077461231082 3.7779175958811595 29.36257430223623

- 라쏘 회귀 모델이 가장 높은 R^2 값과 가장 낮은 오차를 보여 가장 우수한 성능을 나타냄
- 선형회귀와 릿지는 유사한 성능을 보였으며 SGD 회귀 모델은 상대적으로 낮은 예측 성능을 보임



SUMMARY

- 경사하강법 기반 선형회귀 학습 방법
 - 전체 배치 경사하강법 : 전체 데이터를 사용하여 한 번에 기울기를 계산하고 파라미터를 업데이트
 - 확률적 경사하강법 (SGD): 하나의 데이터 샘플을 이용하여 파라미터를 업데이트 하며 빠른 학습이 가능함
 - 미니 배치 경사하강법 : 일정한 크기의 데이터 묶음을 사용하여 기울기를 계산하는 방법으로 학습 안정성과 속도를 모두 고려한 방식
- 과대적합(Overfitting) 해결 방법
 - Ridge 회귀 (L2 정규화): 가중치의 크기를 제한하여 모델의 복잡도를 줄이는 방법
 - Lasso 회귀 (L1 정규화): 일부 가중치를 0으로 만들어 불필요한 특성을 제거하는 효과가 있음
- 실습: 사이킷런(scikit-learn)을 이용하여 Linear Regression, Ridge, Lasso, SGDRegressor 모델을 구현하고 성능을 비교



Q&A

궁금한 사항은 아래 방법으로 문의 바랍니다.



이메일
dana1112@bible.ac.kr



LMS 쪽지

THANK YOU FOR YOUR ATTENTION!

한국성서대학교 | 인공지능융합학부 | 머신러닝